Centro Paula Souza

Faculdade de Tecnologia de Votorantim

Curso de Ciência de Dados para Negócios

Aluno: Ana Elisa Rubinato Cavalheiro

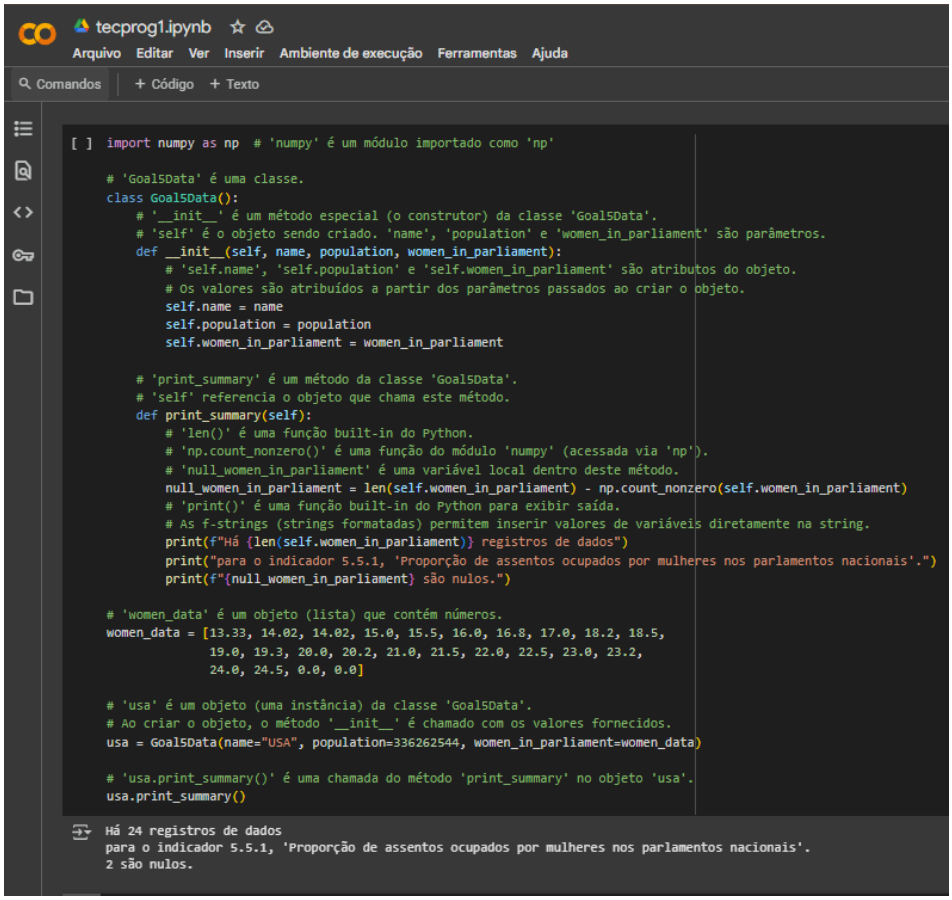
Técnicas de Programação para Ciência de Dados

POO: herança, polimorfismo, abstração e encapsulamento

Instruções

Implemente em Python os conceitos de herança e polimorfismo usando os dados dos assentos de mulheres no parlamento, conta bancária e cursos EAD.

- **Mulheres no Parlamento:**



Fatec Votorantim

Centro Paula Souza

```python
import matplotlib.pyplot as plt  # Importa o módulo para plotagem gráfica
import numpy as np  # Importa o módulo numpy para operações numéricas eficientes
from scipy.stats import linregress  # Importa a função para realizar regressão linear

# 'Goal5Data' class definition (assumindo que esta classe está definida em outro lugar,
# como nos exemplos anteriores, e serve como classe pai)
class Goal5Data():
    def __init__(self, name, population, women_in_parliament):
        self.name = name
        self.population = population
        self.women_in_parliament = women_in_parliament

    def print_summary(self):
        null_women_in_parliament = len(self.women_in_parliament) - np.count_nonzero(self.women_in_parliament)
        print(f"Há {len(self.women_in_parliament)} registros de dados")
        print("para o indicador 5.5.1, 'Proporção de assentos ocupados por mulheres nos parlamentos nacionais'.")
        print(f"{null_women_in_parliament} são nulos.")

# 'Goal5TimeSeries' é uma classe que herda da classe 'Goal5Data'.
class Goal5TimeSeries(Goal5Data):
    # '__init__' é o método construtor da classe 'Goal5TimeSeries'.
    # 'self' referencia a instância do objeto que está sendo criado.
    # 'name', 'population', 'women_in_parliament' são atributos herdados de 'Goal5Data'.
    # 'timestamps' é um novo atributo específico para objetos 'Goal5TimeSeries'.
    def __init__(self, name, population, women_in_parliament, timestamps):
        # 'super().__init__(...)'' chama o método '__init__' da classe pai ('Goal5Data')
        # para inicializar os atributos herdados.
        super().__init__(name, population, women_in_parliament)
        # 'self.timestamps' é um atributo para armazenar a série temporal (anos neste caso).
        self.timestamps = timestamps
        # 'self.slope', 'self.intercept', 'self.r_squared' são atributos para armazenar
        # os resultados da regressão linear, inicializados como None.
        self.slope = None
        self.intercept = None
        self.r_squared = None

    # 'fit_trendline' é um método para calcular a linha de tendência usando regressão linear.
    # 'self' referencia o objeto que chama este método.
    def fit_trendline(self):
        # 'linregress()' realiza a regressão linear nos dados de 'timestamps' (eixo x)
        # e 'self.women_in_parliament' (eixo y). Retorna um objeto com os resultados.
        result = linregress(self.timestamps, self.women_in_parliament)
        # 'self.slope' armazena a inclinação da linha de tendência, arredondada para 3 casas decimais.
        self.slope = round(result.slope, 3)
        # 'self.intercept' armazena o ponto onde a linha de tendência cruza o eixo y.
        self.intercept = result.intercept
        # 'self.r_squared' armazena o coeficiente de determinação (R²), que indica o quão bem a
        # linha de tendência se ajusta aos dados, arredondado para 3 casas decimais.
        self.r_squared = round(result.rvalue**2, 3)
        # Retorna a inclinação e o R² da linha de tendência.
        return self.slope, self.r_squared

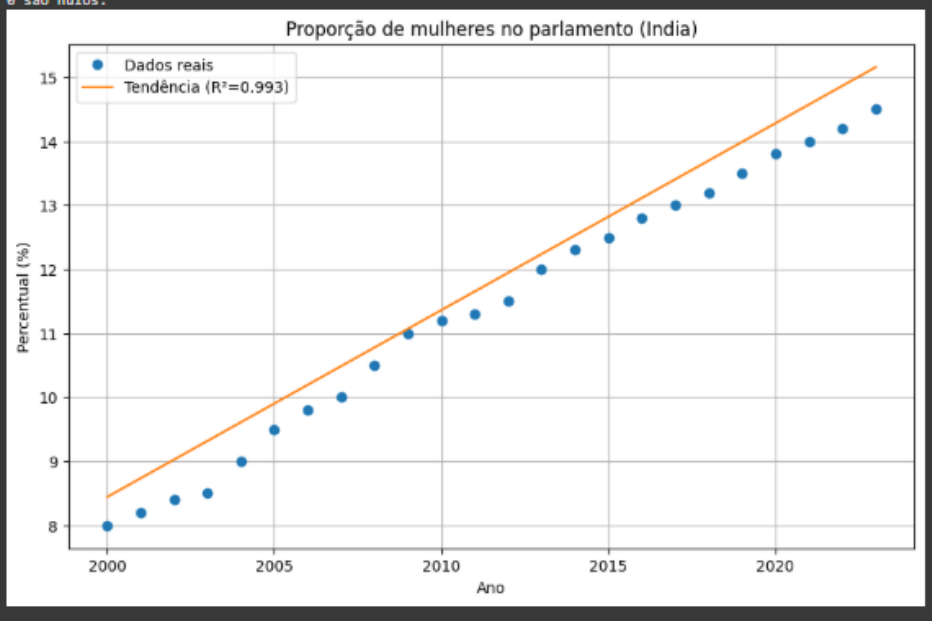
    # 'plot_trendline' é um método para plotar os dados e a linha de tendência.
    # 'self' referencia o objeto que chama este método.
    def plot_trendline(self):
        # Verifica se a linha de tendência já foi ajustada. Se não, chama 'fit_trendline'.
        if self.slope is None or self.intercept is None or self.r_squared is None:
            self.fit_trendline()
        # 'plt.figure()' cria uma nova figura para o gráfico com um tamanho específico.
        plt.figure(figsize=(10, 6))
        # 'plt.plot()' plota os dados reais ('timestamps' no eixo x,
        # 'self.women_in_parliament' no eixo y) com marcadores circulares ('o') e uma legenda.
        plt.plot(self.timestamps, self.women_in_parliament, 'o', label='Dados reais')
        # 'plt.plot()' plota a linha de tendência. Os valores do eixo y são calculados usando
        # a equação da reta (y = mx + c), onde 'm' é a inclinação ('self.slope') e 'c' é o intercepto ('self.intercept').
        plt.plot(self.timestamps, np.array(self.timestamps) * self.slope + self.intercept, '-', label=f'Tendência (R²={self.r_squared})')
        # 'plt.title()' define o título do gráfico, incluindo o nome do país do objeto.
        plt.title(f"Proporção de mulheres no parlamento ({self.name})")
        # 'plt.xlabel()' define o rótulo do eixo x.
        plt.xlabel("Ano")
        # 'plt.ylabel()' define o rótulo do eixo y.
        plt.ylabel("Percentual (%)")
        # 'plt.legend()' exibe a legenda para identificar os dados reais e a linha de tendência.
        plt.legend()
        # 'plt.grid(True)' adiciona uma grade ao gráfico para facilitar a leitura dos valores.
        plt.grid(True)
        # 'plt.show()' exibe o gráfico na tela.
        plt.show()

# Dados simulados para a Índia
# 'timestamps' é um objeto (lista) contendo os anos de 2000 a 2023.
timestamps = list(range(2000, 2024)) # 24 anos
# 'women_data_india' é um objeto (lista) contendo a proporção de mulheres no parlamento da Índia para cada ano.
women_data_india = [8.0, 8.2, 8.4, 8.5, 9.0, 9.5, 9.8, 10.0, 10.5, 11.0,
                    11.2, 11.3, 11.5, 12.0, 12.3, 12.5, 12.8, 13.0, 13.2, 13.5,
                    13.8, 14.0, 14.2, 14.5]

# 'india' é um objeto (uma instância) da classe 'Goal5TimeSeries'.
# Os argumentos passados ao criar o objeto inicializam seus atributos: nome, população,
# dados da proporção de mulheres e os timestamps.
india = Goal5TimeSeries(name="India", population=1400000000, women_in_parliament=women_data_india, timestamps=timestamps)

# Chamando métodos no objeto 'india'
# 'india.print_summary()' chama o método 'print_summary()' herdado da classe 'Goal5Data' para exibir um resumo dos dados.
india.print_summary()
# 'india.plot_trendline()' chama o método 'plot_trendline()' definido na classe 'Goal5TimeSeries' para gerar o gráfico.
india.plot_trendline()
```

```
Há 24 registros de dados
para o indicador 5.5.1, 'Proporção de assentos ocupados por mulheres nos parlamentos nacionais'.
0 são nulos.
```



- **Conta Bancária**

```python
from abc import ABC, abstractmethod # Classe abstrata - só define o que deve ser feito
class ContaBancaria(ABC):
    def __init__(self, titular, saldo=0):
        self.titular = titular
        self._saldo = saldo # atributo protegido

    def ver_saldo(self):
        print(f"Saldo atual: R$ {self._saldo:.2f}")

    @abstractmethod
    def sacar(self, valor):
        pass

    @abstractmethod
    def depositar(self, valor):
        pass

# Implementação concreta - esconde os detalhes
class ContaCorrente(ContaBancaria):
    def sacar(self, valor):
        if valor > self._saldo:
            print("Saldo insuficiente.")
        else:
            self._saldo -= valor
            print(f"Saque de R$ {valor:.2f} realizado com sucesso.")

    def depositar(self, valor):
        self._saldo += valor
        print(f"Depósito de R$ {valor:.2f} realizado com sucesso.")

# Uso
conta = ContaCorrente("João", saldo=500)

conta.ver_saldo() # Mostra o saldo
conta.sacar(200) # Faz um saque
conta.ver_saldo() # Verifica novamente
```

```
Saldo atual: R$ 500.00
Saque de R$ 200.00 realizado com sucesso.
Saldo atual: R$ 300.00
```

- **Cursos EAD**

```
[ ]  class AlunoRegular:
         # Define a classe AlunoRegular
         def entregar(self):
             # Define o método entregar para a classe AlunoRegular
             print("Aluno regular entregou a tarefa pelo portal da escola.")

     class AlunoEAD:
         # Define a classe AlunoEAD
         def entregar(self):
             # Define o método entregar para a classe AlunoEAD
             print("Aluno EAD entregou a tarefa via sistema online.")

     class Professor:
         # Define a classe Professor
         def entregar(self):
             # Define o método entregar para a classe Professor
             print("Professor não entrega tarefa, ele corrige!")

     # Função polimórfica usando duck typing
     def entregar_tarefa(participante):
         # Define uma função que aceita um 'participante' como argumento
         participante.entregar() # Chama o método 'entregar' do objeto participante
                                 # Não importa a classe do objeto, contanto que ele tenha o método 'entregar'!

     # Testando
     # Cria uma lista chamada 'participantes' contendo objetos de diferentes classes
     participantes = [AlunoRegular(), AlunoEAD(), Professor()]

     # Itera sobre cada objeto na lista 'participantes'
     for p in participantes:
         # Chama a função 'entregar_tarefa' passando cada objeto como argumento
         entregar_tarefa(p)
```

```
Aluno regular entregou a tarefa pelo portal da escola.
Aluno EAD entregou a tarefa via sistema online.
Professor não entrega tarefa, ele corrige!
```